UNIVERSIDAD DE MONTERREY

Escuela de Ingeniería y Tecnologías

Ingeniería en Tecnologías Computacionales
Ingeniería Industrial y de Sistemas

Inteligencia Artificial II

Single-Neuron Network With and Without PCA

Dr. Andrés Hernández Gutierréz

ITC | Carlos Ignacio Huerta Carrizales 600291

IIS | Ramón Lerma Hernández 636525

IIS | Bernando Roesch Valdés 614056

San Pedro Garza García, Monterrey, Nuevo León a 05 de marzo de 2026

1. Introduction to the Real-World Problem

El cáncer de mama es uno de los tipos de cáncer más comunes en el mundo, comúnmente se da entre las mujeres y se produce cuando algunas células de la mama comienzan a crecer de manera anormal formando un tumor. Con el tiempo, el crecimiento de estos tumores puede afectar negativamente a todo el cuerpo.

La principal problemática de esta enfermedad es que en las etapas iniciales del cáncer es muy difícil encontrar síntomas evidentes de que se está formando el tumor, debido a esto pueden haber muchos casos que a pesar de que se podrían haber evitado fácilmente con un tratamiento temprano pero que terminan siendo mortales para el paciente. Por esta razón, la detección temprana es de gran importancia para aumentar las probabilidades de tratamiento exitoso y supervivencia. Utilizando métodos de diagnóstico como la mamografía, los ultrasonidos y el análisis de biopsias se pueden identificar las características de los síntomas presentes entre los pacientes.

Los modelos de machine learning nos pueden ayudar a mejorar estos procesos de diagnóstico, siendo estos capaces de análizar los datos obtenidos e identificar patrónes que se repiten en los pacientes que padecen de la enfermedad.

Wisconsin Breast Cancer Dataset

Este Dataset tiene aplicación médica, enfocada en el diágnostico de cáncer de mama. Entre las variables predictivas se encuentran: el radio, la textura, perímetro, área, suavidad, compacidad, concavidad, puntos de concavidad, la simetría y la dimensión fractal. Se calcula la media, la desviación estándar y "peor" (media de los 3 valores más grandes) de cada feature, dando un total de 30. La variable objetivo es Diagnosis (Maligno y Benigno).

Data Cleaning

Es importante manejar los valores faltantes, dependiendo de los datos se puede decidir si es necesario eliminar una columna o reemplazar los valores faltantes con alguna medida de tendencia central.

In [1]:
```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Cargar el dataset (Añadiendo los títulos de las columnas, proveniente de la docum
column_names = ['id', 'diagnosis', 'mean_radius', 'mean_texture', 'mean_perimeter',
```

```
df = pd.read_csv('wdbc.data', names=column_names)

# Limpieza de la columna ID (irrelevante) y mapeo de la columna 'diagnosis' para co
df = df.drop(columns=['id'])
df['diagnosis'] = df['diagnosis'].map({'M': 1, 'B': 0})

# Validación de datos y manejo de valores faltantes
# Conteo por columna y suma de todas las columnas.
if df.isnull().sum().sum():
    print("Existen valores faltantes en el dataset.")
else:
    print("No existen valores faltantes en el dataset.")
```

```
No existen valores faltantes en el dataset.
```

In [5]:
```
df
```

Out[5]:

	diagnosis	mean_radius	mean_texture	mean_perimeter	mean_area	mean_smoothness
0	1	17.99	10.38	122.80	1001.0	0.11840
1	1	20.57	17.77	132.90	1326.0	0.08474
2	1	19.69	21.25	130.00	1203.0	0.10960
3	1	11.42	20.38	77.58	386.1	0.14250
4	1	20.29	14.34	135.10	1297.0	0.10030
...	...	...	...	...	...	...
564	1	21.56	22.39	142.00	1479.0	0.11100
565	1	20.13	28.25	131.20	1261.0	0.09780
566	1	16.60	28.08	108.30	858.1	0.08455
567	1	20.60	29.33	140.10	1265.0	0.11780
568	0	7.76	24.54	47.92	181.0	0.05263

569 rows × 31 columns

Exploratory Data Analysis

In [ ]:
```
df.describe()
```

Out[ ]:

	diagnosis	mean_radius	mean_texture	mean_perimeter	mean_area	mean_smootl
count	569.000000	569.000000	569.000000	569.000000	569.000000	569.0(
mean	0.372583	14.127292	19.289649	91.969033	654.889104	0.09
std	0.483918	3.524049	4.301036	24.298981	351.914129	0.0
min	0.000000	6.981000	9.710000	43.790000	143.500000	0.05
25%	0.000000	11.700000	16.170000	75.170000	420.300000	0.08
50%	0.000000	13.370000	18.840000	86.240000	551.100000	0.09
75%	1.000000	15.780000	21.800000	104.100000	782.700000	0.1(
max	1.000000	28.110000	39.280000	188.500000	2501.000000	0.16

8 rows × 31 columns

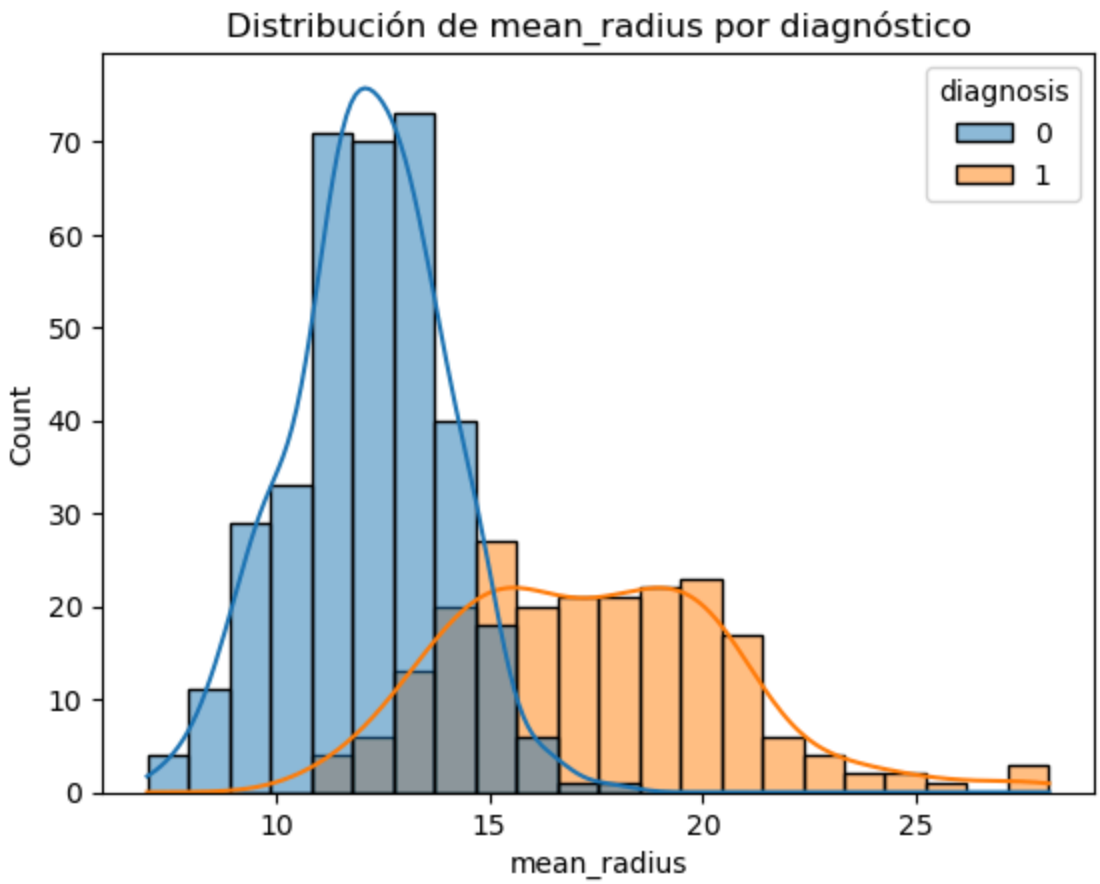
Las escalas de los datos son muy distintas, lo que podría llegar a ocasionar que se asignen valores de relevancia de manera errónea.

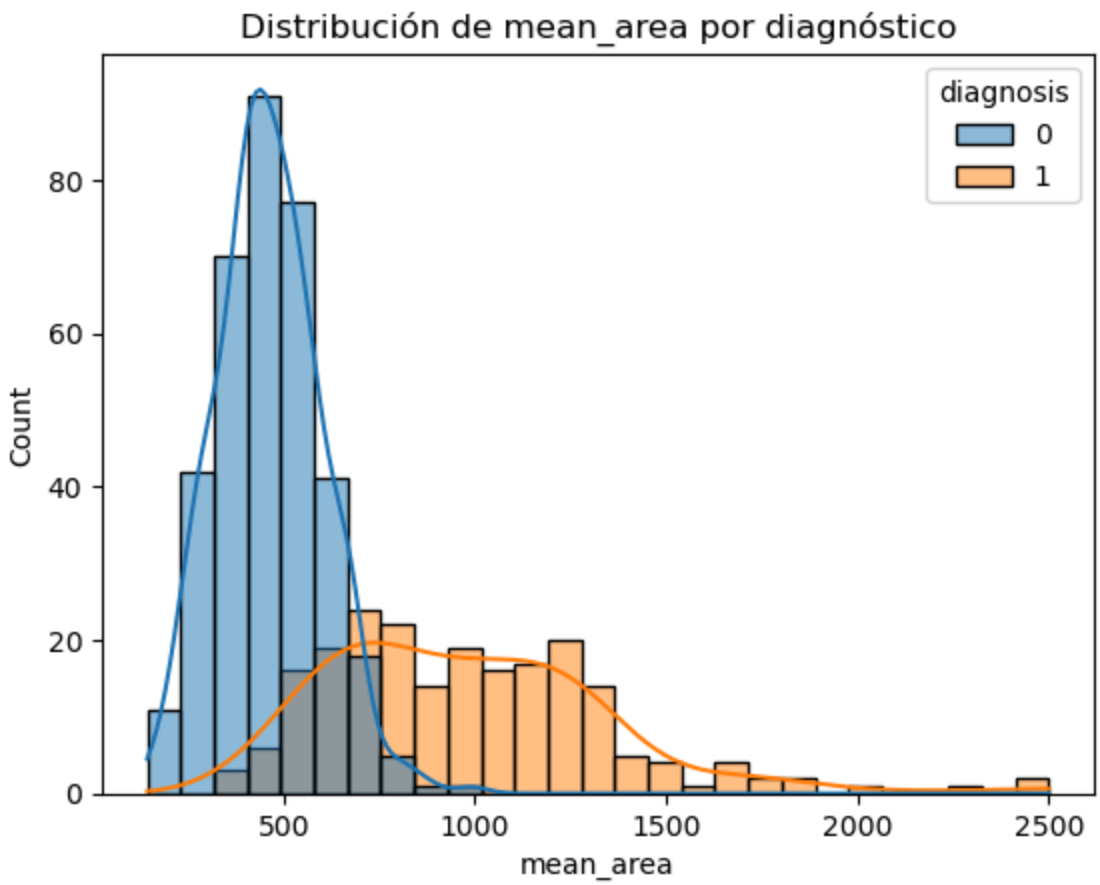
In [3]:
```
df['diagnosis'].value_counts()
```

Out[3]:
```
diagnosis
0    357
1    212
Name: count, dtype: int64
```

Con esto identificamos que de las 569 observaciónes que hay en el dataset 357 estan catalogadas como un diagnóstico negativo del cáncer y 212 como positivo del cáncer. A pesar de que hay un ligero desbalance en los datos estos igual siguén estando lo suficientemente balanceados como para llevar a cabo el modelo.

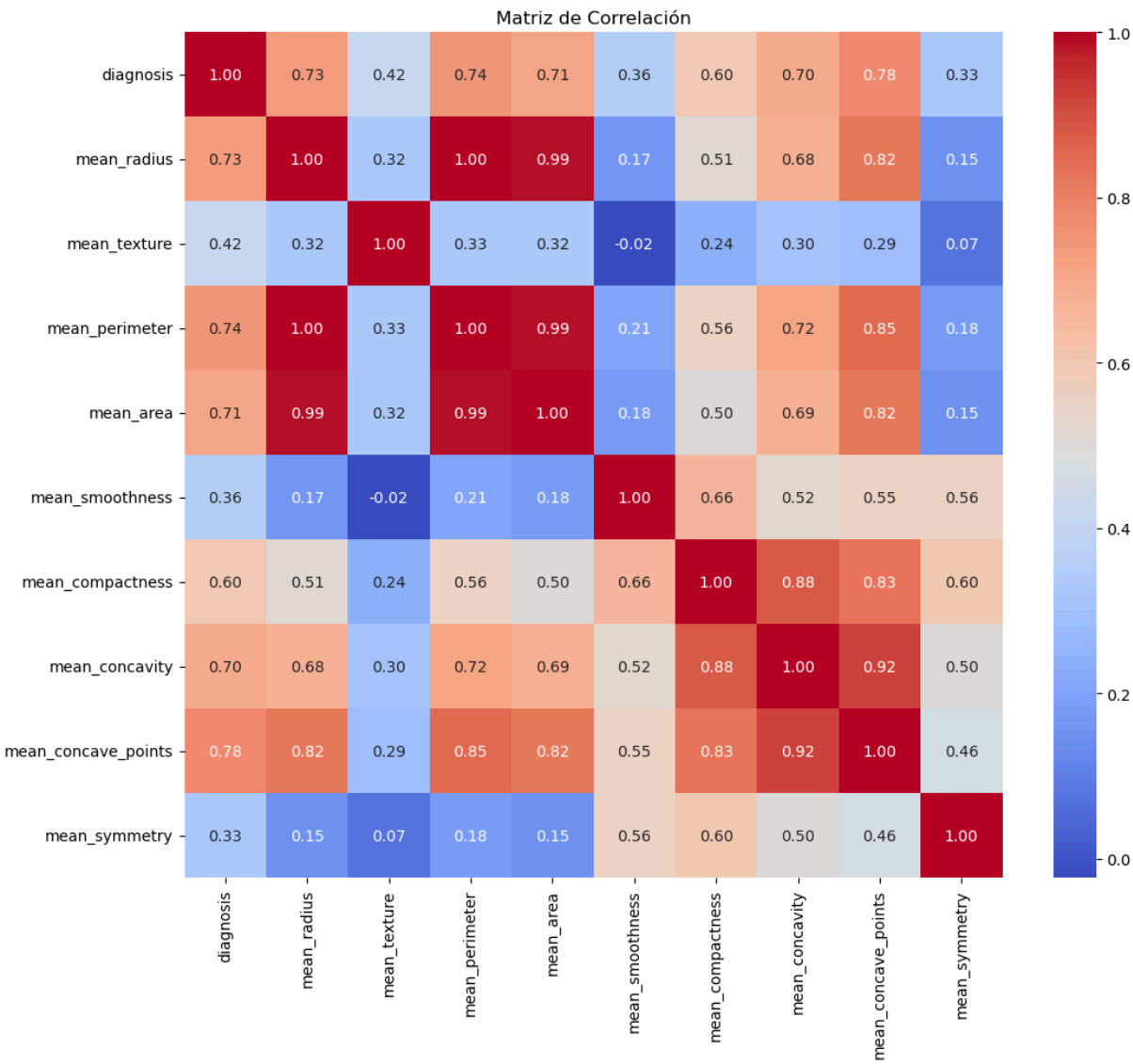
In [ ]:
```
# Histograma de las variables 'mean_radius' y 'mean_area' para observar su distribu
sns.histplot(data = df, x='mean_radius', hue='diagnosis', kde=True)
plt.title('Distribución de mean_radius por diagnóstico')
plt.show()

sns.histplot(data = df, x='mean_area', hue='diagnosis', kde=True)
plt.title('Distribución de mean_area por diagnóstico')
plt.show()
```


Distribución de mean_radius por diagnóstico
diagnosis
0
1
Count
mean_radius
0
10
20
30
40
50
60
70
10
15
20
25


Distribución de mean_area por diagnóstico
diagnosis
0
1
Count
mean_area
0
20
40
60
80
500
1000
1500
2000
2500

Las celúlas cancerígenas muestran una tendencia a tener núcleos más grandes, esto puede indicar un alto valor predictivo de estas variables.

In [ ]:
```
# Análisis de correlación entre las variables para identificar posibles relaciones
correlation_matrix = df.iloc[:, 0:10].corr() # Primeros 10 valores para claridad
plt.figure(figsize=(12, 10))
sns.heatmap(correlation_matrix, annot=True, fmt=".2f", cmap='coolwarm')
plt.title('Matriz de Correlación')
plt.show()
```



Las variables geométricas muestran una alta correlación entre ellas, al ser variables colineales puede ser beneficioso utilizar alguna técnica para reducir la dimensionalidad. Al ser variables geométricas, podría servir obtener una variable derivada que las represente a todas.

Igualmente, es posible notar la alta relación entre compactness, concavity y concave_points, por lo que este sería el segundo grupo de variables que muestran muticolinealidad.

Todas las features de estos grupos muestran una correlación de al menos .7, exceptuando "mean_concavity" con .6, con el diagnóstico. Mostrando un posible alto valor predictivo de

estas variables.

2. Design and Training of the Single-Neuron Model

In [22]:
```
# Importar librerías para la construcción del modelo de red neuronal
import pandas as pd
import plotly.express as px
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np
import tensorflow as tf
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
from sklearn.metrics import precision_score, recall_score, f1_score
from tensorflow.keras.models import load_model
from sklearn.decomposition import PCA
```

In [ ]:
```
# Global seed para reproducibilidad
tf.random.set_seed(4500)

# Preparación de los datos para el modelo
X = df.drop(columns=['diagnosis'])
y = df['diagnosis']

# División de los datos en conjuntos de entrenamiento y prueba
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_sta
```

In [ ]:
```
# Estandarización de las características para mejorar el rendimiento del modelo
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

In [ ]:
```
# Architectura de la red neuronal
model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(X_train_scaled.shape[1],)),
    tf.keras.layers.Dense(1, activation='sigmoid')
])

model.summary()
```

Model: "sequential_4"

Layer (type)	Output Shape	Param #
dense_4 (Dense)	(None, 1)	31

Total params: 31 (124.00 B)

Trainable params: 31 (124.00 B)

```
Non-trainable params: 0 (0.00 B)
```

In [ ]:
```
# Compilación del modelo con el optimizador Adam y la función de pérdida de entropí

model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
```

In [ ]:
```
# Hiperparámetros para el entrenamiento
EPOCHS = 100
LEARNING_RATE = 0.001
BATCH_SIZE = 32
VALIDATION_RATIO = 0.2
```

Descripción de Hiperparámetros:

- EPOCHS: Es el número de veces que se va a repetir el entrenamiento. Esto significa que el modelo repetirá el entrenamiento 100 veces.
- LEARNING_RATE: Determina que tanto van a cambiar los pesos de cada variable en cada entrenamiento.
- BATCH_SIZE: Determina el número de muestras que se usan antes de cambiar los pesos del entrenamiento.
- VALIDATION_RATIO: Es el porcentaje de los datos que se usarán para la validación del modelo tras cada entrenamiento. En este caso se usará el 20% de los datos.

3. Learning Curve Analysis

In [ ]:
```
# Evaluación del modelo antes del entrenamiento
loss, accuracy = model.evaluate(X_test_scaled, y_test)
print(f"Test Accuracy: {accuracy:.4f}")
```

```
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.2544 - loss: 1.1667
Test Accuracy: 0.2544
```

In [ ]:
```
# Entrenamiento del modelo con validación
history = model.fit(X_train_scaled, y_train, epochs=EPOCHS, batch_size=BATCH_SIZE,
```

```
Epoch 1/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.3159 - loss: 1.0307 - val_accur
acy: 0.3736 - val_loss: 0.9370
Epoch 2/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.3599 - loss: 0.9387 - val_accur
acy: 0.4286 - val_loss: 0.8492
Epoch 3/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.4286 - loss: 0.8552 - val_accur
acy: 0.4835 - val_loss: 0.7708
Epoch 4/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.4918 - loss: 0.7804 - val_accur
acy: 0.5714 - val_loss: 0.7021
Epoch 5/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.5659 - loss: 0.7141 - val_accur
acy: 0.6044 - val_loss: 0.6430
Epoch 6/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.6374 - loss: 0.6559 - val_accur
acy: 0.6264 - val_loss: 0.5927
Epoch 7/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7033 - loss: 0.6050 - val_accur
acy: 0.7033 - val_loss: 0.5498
Epoch 8/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.7500 - loss: 0.5605 - val_accur
acy: 0.7473 - val_loss: 0.5131
Epoch 9/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7830 - loss: 0.5216 - val_accur
acy: 0.7912 - val_loss: 0.4814
Epoch 10/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.8159 - loss: 0.4875 - val_accur
acy: 0.8022 - val_loss: 0.4538
Epoch 11/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.8407 - loss: 0.4574 - val_accur
acy: 0.8352 - val_loss: 0.4297
Epoch 12/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.8654 - loss: 0.4307 - val_accur
acy: 0.8571 - val_loss: 0.4084
Epoch 13/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.8764 - loss: 0.4070 - val_accur
acy: 0.8681 - val_loss: 0.3895
Epoch 14/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.8819 - loss: 0.3858 - val_accur
acy: 0.9011 - val_loss: 0.3726
Epoch 15/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.8929 - loss: 0.3667 - val_accur
acy: 0.9011 - val_loss: 0.3574
Epoch 16/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9038 - loss: 0.3495 - val_accur
acy: 0.9121 - val_loss: 0.3438
Epoch 17/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9066 - loss: 0.3339 - val_accur
acy: 0.9121 - val_loss: 0.3314
Epoch 18/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9121 - loss: 0.3198 - val_accur
acy: 0.9231 - val_loss: 0.3201
Epoch 19/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9258 - loss: 0.3069 - val_accur
```

```
acy: 0.9231 - val_loss: 0.3098
Epoch 20/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9368 - loss: 0.2951 - val_accur
acy: 0.9341 - val_loss: 0.3004
Epoch 21/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9423 - loss: 0.2843 - val_accur
acy: 0.9341 - val_loss: 0.2917
Epoch 22/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9423 - loss: 0.2743 - val_accur
acy: 0.9451 - val_loss: 0.2837
Epoch 23/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9451 - loss: 0.2651 - val_accur
acy: 0.9451 - val_loss: 0.2763
Epoch 24/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9505 - loss: 0.2566 - val_accur
acy: 0.9451 - val_loss: 0.2694
Epoch 25/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9533 - loss: 0.2487 - val_accur
acy: 0.9451 - val_loss: 0.2629
Epoch 26/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9533 - loss: 0.2413 - val_accur
acy: 0.9451 - val_loss: 0.2569
Epoch 27/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9560 - loss: 0.2345 - val_accur
acy: 0.9451 - val_loss: 0.2513
Epoch 28/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9560 - loss: 0.2281 - val_accur
acy: 0.9560 - val_loss: 0.2461
Epoch 29/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9588 - loss: 0.2221 - val_accur
acy: 0.9670 - val_loss: 0.2411
Epoch 30/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9615 - loss: 0.2165 - val_accur
acy: 0.9670 - val_loss: 0.2364
Epoch 31/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9615 - loss: 0.2112 - val_accur
acy: 0.9670 - val_loss: 0.2320
Epoch 32/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9615 - loss: 0.2062 - val_accur
acy: 0.9670 - val_loss: 0.2279
Epoch 33/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9615 - loss: 0.2016 - val_accur
acy: 0.9670 - val_loss: 0.2239
Epoch 34/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9670 - loss: 0.1972 - val_accur
acy: 0.9670 - val_loss: 0.2202
Epoch 35/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9670 - loss: 0.1930 - val_accur
acy: 0.9670 - val_loss: 0.2166
Epoch 36/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9670 - loss: 0.1890 - val_accur
acy: 0.9670 - val_loss: 0.2132
Epoch 37/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9670 - loss: 0.1853 - val_accur
acy: 0.9670 - val_loss: 0.2100
Epoch 38/100
```

```
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9670 - loss: 0.1817 - val_accur
acy: 0.9670 - val_loss: 0.2069
Epoch 39/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9670 - loss: 0.1783 - val_accur
acy: 0.9670 - val_loss: 0.2040
Epoch 40/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9670 - loss: 0.1751 - val_accur
acy: 0.9670 - val_loss: 0.2012
Epoch 41/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9698 - loss: 0.1721 - val_accur
acy: 0.9670 - val_loss: 0.1985
Epoch 42/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9698 - loss: 0.1691 - val_accur
acy: 0.9780 - val_loss: 0.1960
Epoch 43/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9725 - loss: 0.1664 - val_accur
acy: 0.9780 - val_loss: 0.1935
Epoch 44/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9725 - loss: 0.1637 - val_accur
acy: 0.9780 - val_loss: 0.1911
Epoch 45/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9725 - loss: 0.1612 - val_accur
acy: 0.9780 - val_loss: 0.1889
Epoch 46/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.9725 - loss: 0.1587 - val_accu
racy: 0.9780 - val_loss: 0.1867
Epoch 47/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9725 - loss: 0.1564 - val_accur
acy: 0.9780 - val_loss: 0.1846
Epoch 48/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9725 - loss: 0.1541 - val_accur
acy: 0.9780 - val_loss: 0.1826
Epoch 49/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9725 - loss: 0.1520 - val_accur
acy: 0.9780 - val_loss: 0.1806
Epoch 50/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9725 - loss: 0.1499 - val_accur
acy: 0.9780 - val_loss: 0.1788
Epoch 51/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 0.9725 - loss: 0.1479 - val_accu
racy: 0.9780 - val_loss: 0.1770
Epoch 52/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9725 - loss: 0.1460 - val_accur
acy: 0.9780 - val_loss: 0.1752
Epoch 53/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9753 - loss: 0.1442 - val_accur
acy: 0.9780 - val_loss: 0.1736
Epoch 54/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9753 - loss: 0.1424 - val_accur
acy: 0.9780 - val_loss: 0.1719
Epoch 55/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9753 - loss: 0.1407 - val_accur
acy: 0.9780 - val_loss: 0.1704
Epoch 56/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9753 - loss: 0.1391 - val_accur
acy: 0.9780 - val_loss: 0.1689
```

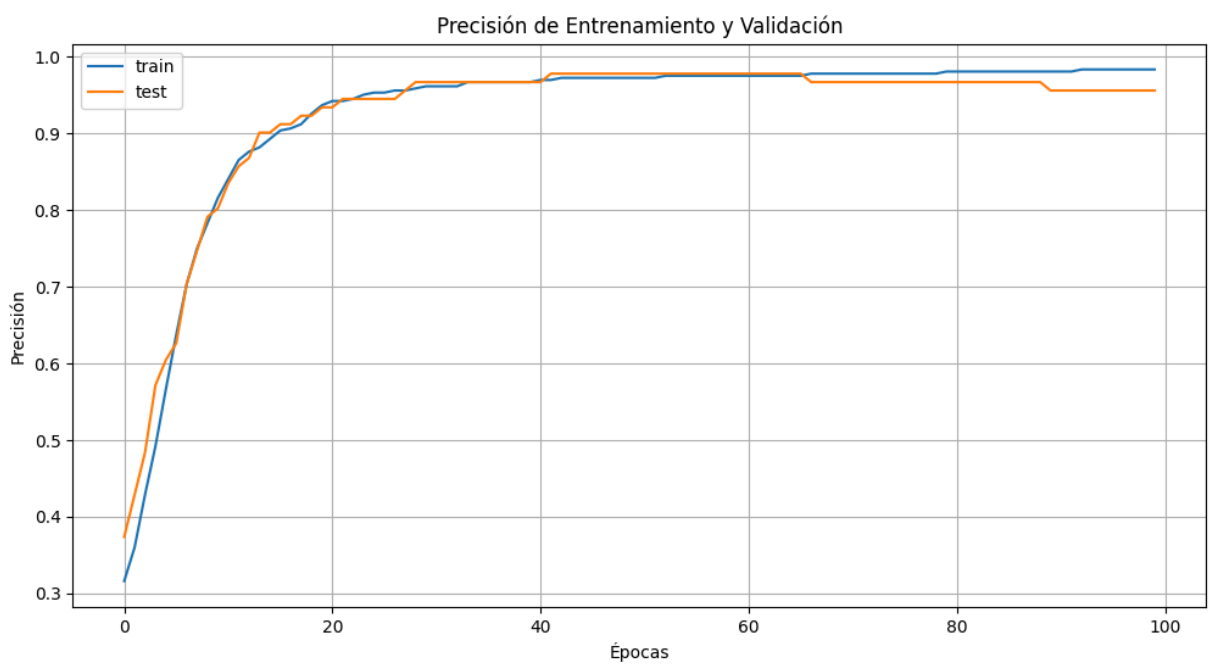
```
Epoch 57/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9753 - loss: 0.1375 - val_accur
acy: 0.9780 - val_loss: 0.1674
Epoch 58/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9753 - loss: 0.1359 - val_accur
acy: 0.9780 - val_loss: 0.1660
Epoch 59/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9753 - loss: 0.1345 - val_accur
acy: 0.9780 - val_loss: 0.1646
Epoch 60/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9753 - loss: 0.1330 - val_accur
acy: 0.9780 - val_loss: 0.1633
Epoch 61/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9753 - loss: 0.1316 - val_accur
acy: 0.9780 - val_loss: 0.1620
Epoch 62/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9753 - loss: 0.1303 - val_accur
acy: 0.9780 - val_loss: 0.1608
Epoch 63/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9753 - loss: 0.1290 - val_accur
acy: 0.9780 - val_loss: 0.1596
Epoch 64/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9753 - loss: 0.1277 - val_accur
acy: 0.9780 - val_loss: 0.1584
Epoch 65/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9753 - loss: 0.1265 - val_accur
acy: 0.9780 - val_loss: 0.1573
Epoch 66/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9753 - loss: 0.1253 - val_accur
acy: 0.9780 - val_loss: 0.1562
Epoch 67/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9780 - loss: 0.1241 - val_accur
acy: 0.9670 - val_loss: 0.1551
Epoch 68/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9780 - loss: 0.1230 - val_accur
acy: 0.9670 - val_loss: 0.1541
Epoch 69/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9780 - loss: 0.1219 - val_accur
acy: 0.9670 - val_loss: 0.1531
Epoch 70/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9780 - loss: 0.1208 - val_accur
acy: 0.9670 - val_loss: 0.1521
Epoch 71/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9780 - loss: 0.1198 - val_accur
acy: 0.9670 - val_loss: 0.1511
Epoch 72/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9780 - loss: 0.1188 - val_accur
acy: 0.9670 - val_loss: 0.1502
Epoch 73/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9780 - loss: 0.1178 - val_accur
acy: 0.9670 - val_loss: 0.1493
Epoch 74/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9780 - loss: 0.1169 - val_accur
acy: 0.9670 - val_loss: 0.1484
Epoch 75/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9780 - loss: 0.1159 - val_accur
```

```
acy: 0.9670 - val_loss: 0.1476
Epoch 76/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9780 - loss: 0.1150 - val_accur
acy: 0.9670 - val_loss: 0.1467
Epoch 77/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9780 - loss: 0.1141 - val_accur
acy: 0.9670 - val_loss: 0.1459
Epoch 78/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9780 - loss: 0.1133 - val_accur
acy: 0.9670 - val_loss: 0.1451
Epoch 79/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9780 - loss: 0.1124 - val_accur
acy: 0.9670 - val_loss: 0.1444
Epoch 80/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9808 - loss: 0.1116 - val_accur
acy: 0.9670 - val_loss: 0.1436
Epoch 81/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9808 - loss: 0.1108 - val_accur
acy: 0.9670 - val_loss: 0.1429
Epoch 82/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9808 - loss: 0.1100 - val_accur
acy: 0.9670 - val_loss: 0.1422
Epoch 83/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9808 - loss: 0.1093 - val_accur
acy: 0.9670 - val_loss: 0.1415
Epoch 84/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9808 - loss: 0.1085 - val_accur
acy: 0.9670 - val_loss: 0.1408
Epoch 85/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9808 - loss: 0.1078 - val_accur
acy: 0.9670 - val_loss: 0.1401
Epoch 86/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9808 - loss: 0.1071 - val_accur
acy: 0.9670 - val_loss: 0.1395
Epoch 87/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9808 - loss: 0.1064 - val_accur
acy: 0.9670 - val_loss: 0.1388
Epoch 88/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9808 - loss: 0.1057 - val_accur
acy: 0.9670 - val_loss: 0.1382
Epoch 89/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9808 - loss: 0.1050 - val_accur
acy: 0.9670 - val_loss: 0.1376
Epoch 90/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9808 - loss: 0.1044 - val_accur
acy: 0.9560 - val_loss: 0.1370
Epoch 91/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9808 - loss: 0.1037 - val_accur
acy: 0.9560 - val_loss: 0.1364
Epoch 92/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9808 - loss: 0.1031 - val_accur
acy: 0.9560 - val_loss: 0.1358
Epoch 93/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9835 - loss: 0.1025 - val_accur
acy: 0.9560 - val_loss: 0.1353
Epoch 94/100
```

```
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9835 - loss: 0.1019 - val_accur
acy: 0.9560 - val_loss: 0.1347
Epoch 95/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9835 - loss: 0.1013 - val_accur
acy: 0.9560 - val_loss: 0.1342
Epoch 96/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9835 - loss: 0.1007 - val_accur
acy: 0.9560 - val_loss: 0.1337
Epoch 97/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9835 - loss: 0.1001 - val_accur
acy: 0.9560 - val_loss: 0.1332
Epoch 98/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9835 - loss: 0.0996 - val_accur
acy: 0.9560 - val_loss: 0.1327
Epoch 99/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9835 - loss: 0.0990 - val_accur
acy: 0.9560 - val_loss: 0.1322
Epoch 100/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9835 - loss: 0.0985 - val_accur
acy: 0.9560 - val_loss: 0.1317
```

In [ ]:

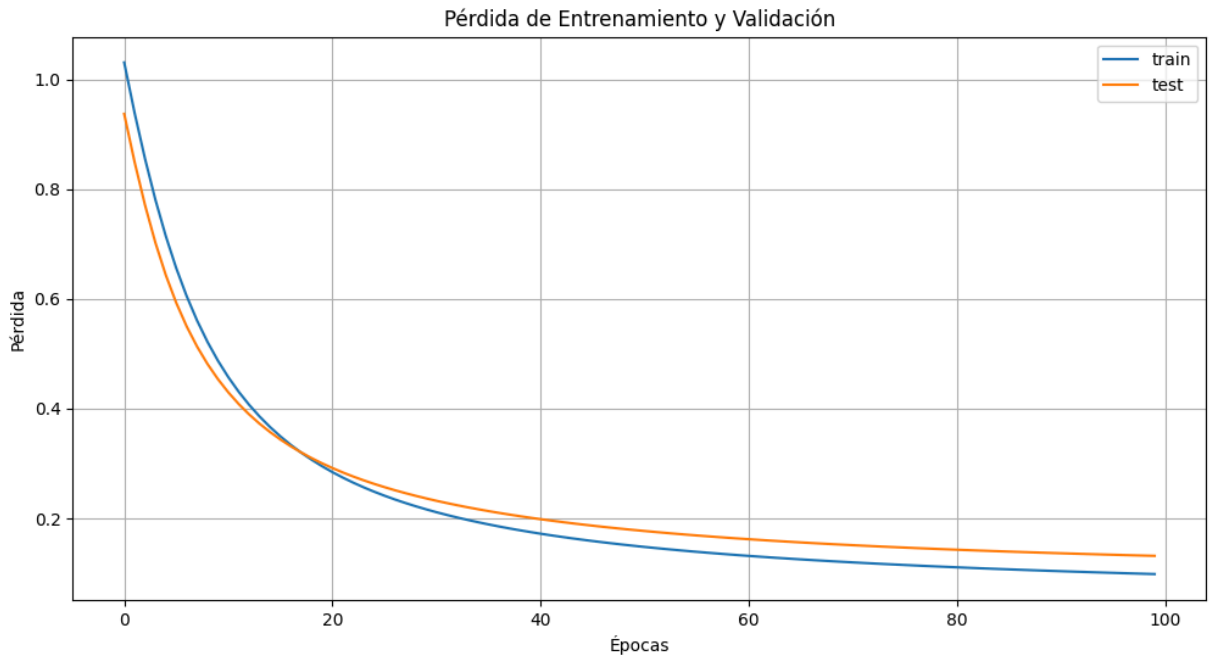
```python
# Gráfica de precisión de entrenamiento y validación a lo largo de las épocas
plt.figure(figsize=(12, 6))
plt.plot(history.history['accuracy'], label='Precisión de Entrenamiento')
plt.plot(history.history['val_accuracy'], label='Precisión de Validación')
plt.title('Precisión de Entrenamiento y Validación')
plt.xlabel('Épocas')
plt.ylabel('Precisión')
plt.legend(['train', 'test'])
plt.grid()
```



In [ ]:

```python
# Gráfica de pérdida de entrenamiento y validación a lo largo de las épocas
plt.figure(figsize=(12, 6))
plt.plot(history.history['loss'], label='Pérdida de Entrenamiento')
plt.plot(history.history['val_loss'], label='Pérdida de Validación')
```

```
plt.title('Pérdida de Entrenamiento y Validación')
plt.xlabel('Épocas')
plt.ylabel('Pérdida')
plt.legend(['train', 'test'])
plt.grid()
```



Interpretación de las curvas

Las curvas muestran que la precisión de entrenamiento y validación aumentan y se estabilizan en valores altos (0.97–0.98), manteniéndose muy cercanas entre sí. Además, la pérdida disminuye de forma constante en ambos conjuntos, con tendencias similares.

Esto indica que no hay underfitting, ya que el modelo alcanza alta precisión, ni overfitting, porque la diferencia entre entrenamiento y validación es mínima. Por lo tanto, el modelo presenta appropriate fitting, ya que aprende correctamente y generaliza bien a los datos de validación.

4. Model Evaluation and Testing Data

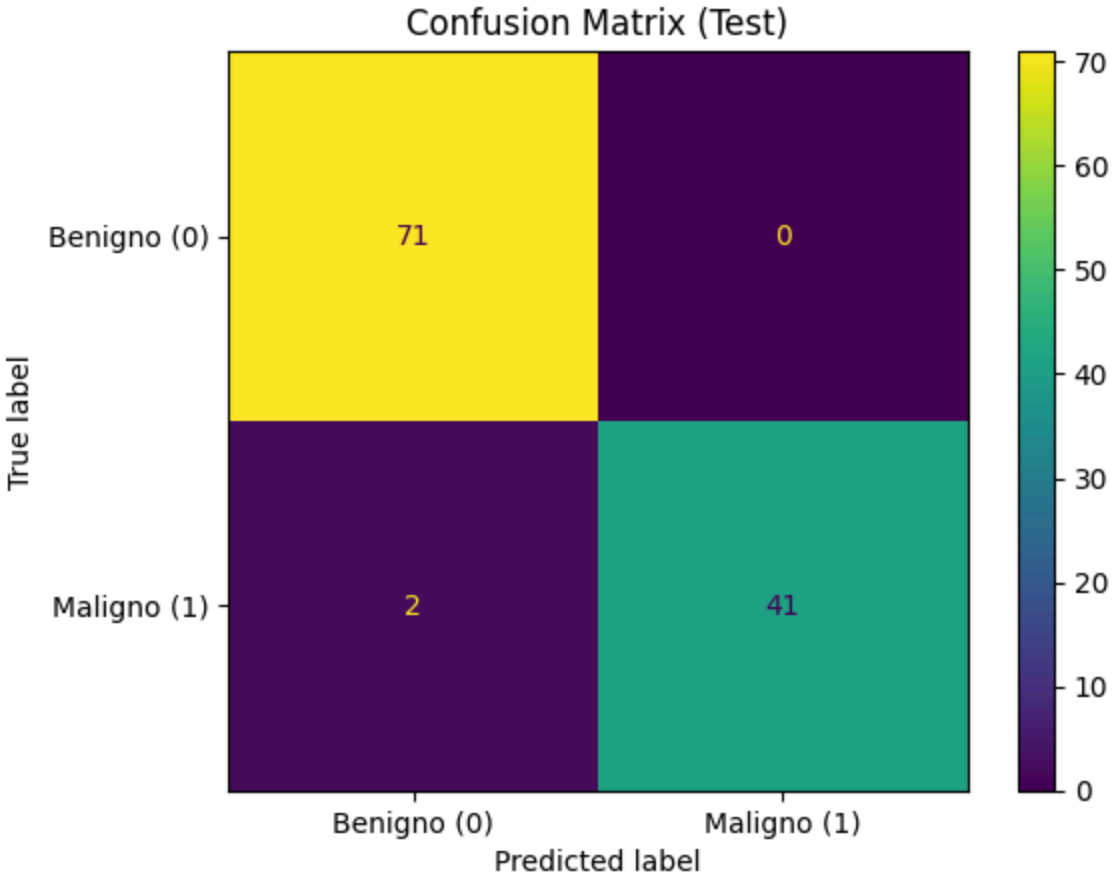
In [ ]:
```
#Predecir (probabilidades) y convertir a clases (0 o 1)
y_prob = model.predict(X_test_scaled)          # probabilidad de clase 1
y_pred = (y_prob >= 0.5).astype(int).ravel()   # umbral 0.5 -> clase
```

```
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step
```

In [ ]:
```
#Matriz de confusión
cm = confusion_matrix(y_test, y_pred)
print("\nConfusion matrix:\n", cm)

disp = ConfusionMatrixDisplay(cm, display_labels=["Benigno (0)", "Maligno (1)"])
```

```
disp.plot(values_format="d")
plt.title("Confusion Matrix (Test)")
plt.show()
```

```
Confusion matrix:
 [[71  0]
 [ 2 41]]
```



La matriz de confusión muestra que el modelo clasificó correctamente 71 casos benignos y 42 malignos, con solo dos errores, donde un tumor maligno fue clasificado como benigno. Esto indica un desempeño muy alto del modelo en el conjunto de prueba.

In [ ]:
```
#Métricas
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

TN, FP, FN, TP = cm.ravel()
specificity = TN / (TN + FP)

print("\nPrecision:", round(precision, 4))
print("Recall:", round(recall, 4))
print("Specificity:", round(specificity, 4))
print("F1-score:", round(f1, 4))
```

```
Precision: 1.0
Recall: 0.9535
Specificity: 1.0
F1-score: 0.9762
```

Interpretación de las métricas

- La precision = 1.0 indica que todos los casos que el modelo clasificó como malignos realmente lo eran. Esto significa que el modelo no generó falsos positivos, es decir, ningún tumor benigno fue clasificado incorrectamente como maligno.
- El recall = 0.9535 mide la capacidad del modelo para identificar correctamente los casos malignos. Este valor indica que el modelo detecta aproximadamente el 95.4% de los tumores malignos, lo que significa que solo un caso maligno fue clasificado incorrectamente como benigno (falso negativo).
- La specificity = 1.0 indica que todos los tumores benignos fueron correctamente clasificados como benignos. Esto muestra que el modelo tiene una excelente capacidad para distinguir los casos no cancerígenos.
- El F1-score = 0.9762 combina precision y recall en una sola métrica, proporcionando una medida del equilibrio entre ambas. Un valor cercano a 1 indica que el modelo tiene un muy buen rendimiento general en la clasificación.

En el contexto del diagnóstico de cáncer de mama, la métrica más importante es recall (sensibilidad), ya que un falso negativo implicaría que un tumor maligno no sea detectado, lo que podría retrasar el tratamiento del paciente. Por lo tanto, es fundamental que el modelo identifique correctamente la mayor cantidad posible de casos malignos.

En general, el modelo presenta un desempeño muy alto, con métricas cercanas a 1 y solo dos errores de clasificación en el conjunto de prueba. Esto indica que el modelo tiene una muy buena capacidad para diferenciar entre tumores benignos y malignos, por lo que su rendimiento puede considerarse muy adecuado para este problema de clasificación.

5. Save the Model

In [ ]:
```
# Guardar el modelo entrenado
model.save("breast_cancer_model.keras")

print("Modelo guardado correctamente")
```

```
Modelo guardado correctamente
```

In [ ]:
```
# Cargar el modelo guardado
loaded_model = load_model("breast_cancer_model.keras")

print("Modelo cargado correctamente")
```

```
Modelo cargado correctamente
```

In [ ]:
```
loss, accuracy = loaded_model.evaluate(X_test_scaled, y_test)

print("Accuracy del modelo cargado:", accuracy)
```

```
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9825 - loss: 0.0961
Accuracy del modelo cargado: 0.9824561476707458
```

Guardar un modelo entrenado permite reutilizarlo sin tener que volver a entrenarlo, lo que ahorra tiempo y recursos computacionales. Además, facilita su implementación en aplicaciones reales y permite compartirlo o reproducir los resultados en otros sistemas.

6. Apply PCA to the Original Dataset

In [7]:
```
# Cargar el dataset para PCA
dfPCA = pd.read_csv('wdbc.data', names=column_names)

# Limpieza de la columna ID (irrelevante) y mapeo de la columna 'diagnosis' para co
dfPCA = dfPCA.drop(columns=['id'])
dfPCA['diagnosis'] = dfPCA['diagnosis'].map({'M': 1, 'B': 0})
```

In [8]:
```
# Estandarización de las características para PCA
XPCA = dfPCA.drop(columns=['diagnosis'])
yPCA = dfPCA['diagnosis']
XPCA_scaled = StandardScaler().fit_transform(XPCA)

# Matriz de covarianza
cov_matrix = np.cov(XPCA_scaled, rowvar=False)

cov_df = pd.DataFrame(cov_matrix, index=XPCA.columns, columns=XPCA.columns)
```

In [9]:
```
# Eigenvalores y eigenvectores
eigenvalues, eigenvectors = np.linalg.eig(cov_matrix)
print("Eigenvalores:\n", eigenvalues)
```

```
Eigenvalores:
 [1.33049908e+01 5.70137460e+00 2.82291016e+00 1.98412752e+00
 1.65163324e+00 1.20948224e+00 6.76408882e-01 4.77456255e-01
 4.17628782e-01 3.51310875e-01 2.94433153e-01 2.61621161e-01
 2.41782421e-01 1.57286149e-01 9.43006956e-02 8.00034045e-02
 5.95036135e-02 5.27114222e-02 4.95647002e-02 1.33279057e-04
 7.50121413e-04 1.59213600e-03 6.91261258e-03 8.19203712e-03
 1.55085271e-02 1.80867940e-02 2.43836914e-02 2.74877113e-02
 3.12142606e-02 3.00256631e-02]
```

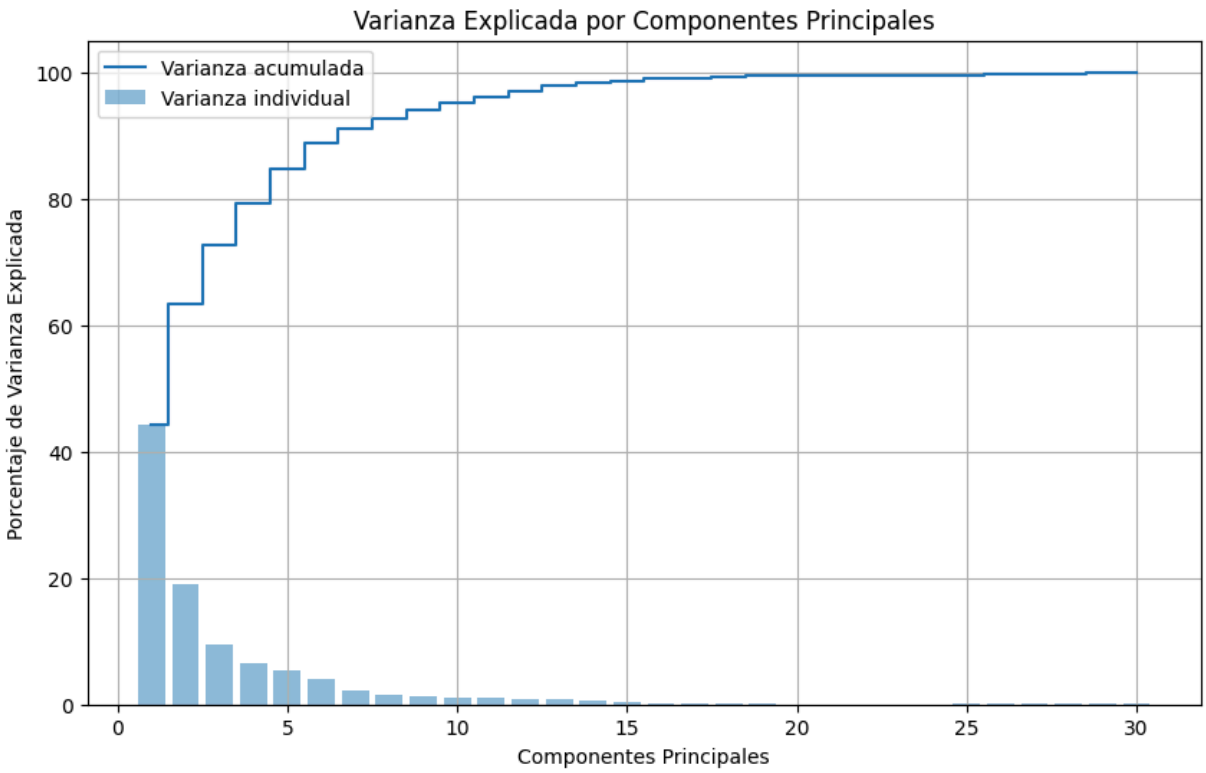
In [10]:
```
# Varianza explicada en porcentaje
explained_variance = eigenvalues / np.sum(eigenvalues) * 100
accumulated_variance = 0

# Hasta alcanzar 95% de la varianza explicada
for i, val in enumerate(explained_variance):
    if accumulated_variance < 95:
```

```
        accumulated_variance = np.sum(explained_variance[:i+1])
        print(f"Componente {i+1}: {val:.2f}% de la varianza explicada, Varianza exp
        k = i + 1
```

```
Componente 1: 44.27% de la varianza explicada, Varianza explicada: 44.27%
Componente 2: 18.97% de la varianza explicada, Varianza explicada: 63.24%
Componente 3: 9.39% de la varianza explicada, Varianza explicada: 72.64%
Componente 4: 6.60% de la varianza explicada, Varianza explicada: 79.24%
Componente 5: 5.50% de la varianza explicada, Varianza explicada: 84.73%
Componente 6: 4.02% de la varianza explicada, Varianza explicada: 88.76%
Componente 7: 2.25% de la varianza explicada, Varianza explicada: 91.01%
Componente 8: 1.59% de la varianza explicada, Varianza explicada: 92.60%
Componente 9: 1.39% de la varianza explicada, Varianza explicada: 93.99%
Componente 10: 1.17% de la varianza explicada, Varianza explicada: 95.16%
```

In [11]:

```
# explained variance ratio plot
plt.figure(figsize=(10, 6))
plt.bar(range(1, len(explained_variance) + 1), explained_variance, alpha=0.5, align
plt.step(range(1, len(explained_variance) + 1), np.cumsum(explained_variance), wher
plt.xlabel('Componentes Principales')
plt.ylabel('Porcentaje de Varianza Explicada')
plt.title('Varianza Explicada por Componentes Principales')
plt.legend()
plt.grid()
```



La gráfica muestra que con aproximadamente 10 componentes principales se explica cerca del 95% de la varianza del dataset. Esto significa que la dimensionalidad se redujo de 30 variables originales a 10 componentes, manteniendo la mayor parte de la información.

Esto también indica que existe redundancia en los datos, ya que muchas variables están correlacionadas y pueden representarse mediante un menor número de componentes

principales.

7. Train a New Single-Neuron Model Using PCA Features

In [19]:
```
#Preparación de los datos
X = dfPCA.drop(columns=['diagnosis'])
y = dfPCA['diagnosis']

#División de lo datos
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

In [20]:
```
# Estandarización de las características para mejorar el rendimiento del modelo
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

In [23]:
```
pca = PCA(n_components=10)
X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)

print("Train PCA shape:", X_train_pca.shape)
print("Test PCA shape:", X_test_pca.shape)
```

```
Train PCA shape: (455, 10)
Test PCA shape: (114, 10)
```

In [25]:
```
tf.random.set_seed(4500)

# Hiperparámetros para el entrenamiento
EPOCHS = 100
BATCH_SIZE = 32
VALIDATION_RATIO = 0.2
```

In [26]:
```
#Arquitectura del modelo
model_pca = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(X_train_pca.shape[1],)),
    tf.keras.layers.Dense(1, activation='sigmoid')
])

# Compilación del modelo con el optimizador Adam y la función de pérdida de entropí
model_pca.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'

#Entrenamiento del modelo
history_pca = model_pca.fit(
    X_train_pca, y_train,
    epochs=EPOCHS,
    batch_size=BATCH_SIZE,
    validation_split=VALIDATION_RATIO,
```

```
    verbose=1
)
```

```
Epoch 1/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 2s 37ms/step - accuracy: 0.5431 - loss: 0.8378 - val_accu
racy: 0.5275 - val_loss: 0.8938
Epoch 2/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 0.5519 - loss: 0.8137 - val_accu
racy: 0.5385 - val_loss: 0.8681
Epoch 3/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.5579 - loss: 0.7909 - val_accu
racy: 0.5714 - val_loss: 0.8428
Epoch 4/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - accuracy: 0.5684 - loss: 0.7687 - val_accu
racy: 0.5714 - val_loss: 0.8182
Epoch 5/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - accuracy: 0.5698 - loss: 0.7472 - val_accu
racy: 0.5824 - val_loss: 0.7943
Epoch 6/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 0.5799 - loss: 0.7264 - val_accu
racy: 0.5934 - val_loss: 0.7711
Epoch 7/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.5966 - loss: 0.7062 - val_accu
racy: 0.5934 - val_loss: 0.7486
Epoch 8/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.6042 - loss: 0.6868 - val_accu
racy: 0.6264 - val_loss: 0.7267
Epoch 9/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.6081 - loss: 0.6679 - val_accur
acy: 0.6484 - val_loss: 0.7055
Epoch 10/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.6099 - loss: 0.6497 - val_accur
acy: 0.6813 - val_loss: 0.6850
Epoch 11/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.6172 - loss: 0.6321 - val_accur
acy: 0.7033 - val_loss: 0.6651
Epoch 12/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.6195 - loss: 0.6152 - val_accur
acy: 0.7033 - val_loss: 0.6458
Epoch 13/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.6222 - loss: 0.5988 - val_accur
acy: 0.7143 - val_loss: 0.6272
Epoch 14/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.6574 - loss: 0.5830 - val_accur
acy: 0.7253 - val_loss: 0.6092
Epoch 15/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.6674 - loss: 0.5678 - val_accur
acy: 0.7473 - val_loss: 0.5919
Epoch 16/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.6835 - loss: 0.5531 - val_accur
acy: 0.7473 - val_loss: 0.5751
Epoch 17/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.7036 - loss: 0.5390 - val_accur
acy: 0.7582 - val_loss: 0.5589
Epoch 18/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7295 - loss: 0.5254 - val_accur
acy: 0.7692 - val_loss: 0.5433
Epoch 19/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7493 - loss: 0.5123 - val_accur
```

```
acy: 0.7912 - val_loss: 0.5283
Epoch 20/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.7684 - loss: 0.4996 - val_accur
acy: 0.7912 - val_loss: 0.5138
Epoch 21/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7782 - loss: 0.4875 - val_accur
acy: 0.7912 - val_loss: 0.4998
Epoch 22/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.7782 - loss: 0.4758 - val_accur
acy: 0.7912 - val_loss: 0.4864
Epoch 23/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.7864 - loss: 0.4646 - val_accur
acy: 0.7912 - val_loss: 0.4734
Epoch 24/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.7926 - loss: 0.4537 - val_accur
acy: 0.8022 - val_loss: 0.4610
Epoch 25/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 0.8045 - loss: 0.4433 - val_accu
racy: 0.8022 - val_loss: 0.4491
Epoch 26/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 0.8185 - loss: 0.4333 - val_accu
racy: 0.8132 - val_loss: 0.4376
Epoch 27/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 0.8200 - loss: 0.4236 - val_accu
racy: 0.8352 - val_loss: 0.4266
Epoch 28/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 0.8200 - loss: 0.4144 - val_accu
racy: 0.8352 - val_loss: 0.4161
Epoch 29/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.8346 - loss: 0.4054 - val_accu
racy: 0.8352 - val_loss: 0.4060
Epoch 30/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.8381 - loss: 0.3968 - val_accu
racy: 0.8352 - val_loss: 0.3964
Epoch 31/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 0.8444 - loss: 0.3885 - val_accu
racy: 0.8462 - val_loss: 0.3872
Epoch 32/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.8455 - loss: 0.3806 - val_accu
racy: 0.8462 - val_loss: 0.3784
Epoch 33/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 0.8455 - loss: 0.3729 - val_accu
racy: 0.8462 - val_loss: 0.3700
Epoch 34/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.8535 - loss: 0.3655 - val_accu
racy: 0.8571 - val_loss: 0.3620
Epoch 35/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.8550 - loss: 0.3584 - val_accu
racy: 0.8571 - val_loss: 0.3543
Epoch 36/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.8620 - loss: 0.3515 - val_accur
acy: 0.8571 - val_loss: 0.3471
Epoch 37/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.8642 - loss: 0.3449 - val_accur
acy: 0.8571 - val_loss: 0.3402
Epoch 38/100
```

```
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.8772 - loss: 0.3385 - val_accur
acy: 0.8571 - val_loss: 0.3336
Epoch 39/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.8899 - loss: 0.3324 - val_accur
acy: 0.8571 - val_loss: 0.3273
Epoch 40/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.8939 - loss: 0.3265 - val_accur
acy: 0.8571 - val_loss: 0.3213
Epoch 41/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.8939 - loss: 0.3207 - val_accur
acy: 0.8571 - val_loss: 0.3156
Epoch 42/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.8939 - loss: 0.3152 - val_accur
acy: 0.8571 - val_loss: 0.3102
Epoch 43/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.8975 - loss: 0.3099 - val_accur
acy: 0.8571 - val_loss: 0.3050
Epoch 44/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9028 - loss: 0.3047 - val_accur
acy: 0.8571 - val_loss: 0.3001
Epoch 45/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9028 - loss: 0.2998 - val_accur
acy: 0.8681 - val_loss: 0.2953
Epoch 46/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9032 - loss: 0.2950 - val_accur
acy: 0.8681 - val_loss: 0.2908
Epoch 47/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9032 - loss: 0.2903 - val_accur
acy: 0.8681 - val_loss: 0.2865
Epoch 48/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9032 - loss: 0.2858 - val_accur
acy: 0.8681 - val_loss: 0.2824
Epoch 49/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 0.9032 - loss: 0.2815 - val_accu
racy: 0.8791 - val_loss: 0.2785
Epoch 50/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9032 - loss: 0.2773 - val_accur
acy: 0.8901 - val_loss: 0.2748
Epoch 51/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9032 - loss: 0.2732 - val_accur
acy: 0.9011 - val_loss: 0.2712
Epoch 52/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9032 - loss: 0.2693 - val_accur
acy: 0.9011 - val_loss: 0.2678
Epoch 53/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9073 - loss: 0.2655 - val_accur
acy: 0.9011 - val_loss: 0.2645
Epoch 54/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9137 - loss: 0.2618 - val_accur
acy: 0.9011 - val_loss: 0.2613
Epoch 55/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9267 - loss: 0.2582 - val_accur
acy: 0.9011 - val_loss: 0.2583
Epoch 56/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9307 - loss: 0.2547 - val_accur
acy: 0.9121 - val_loss: 0.2554
```

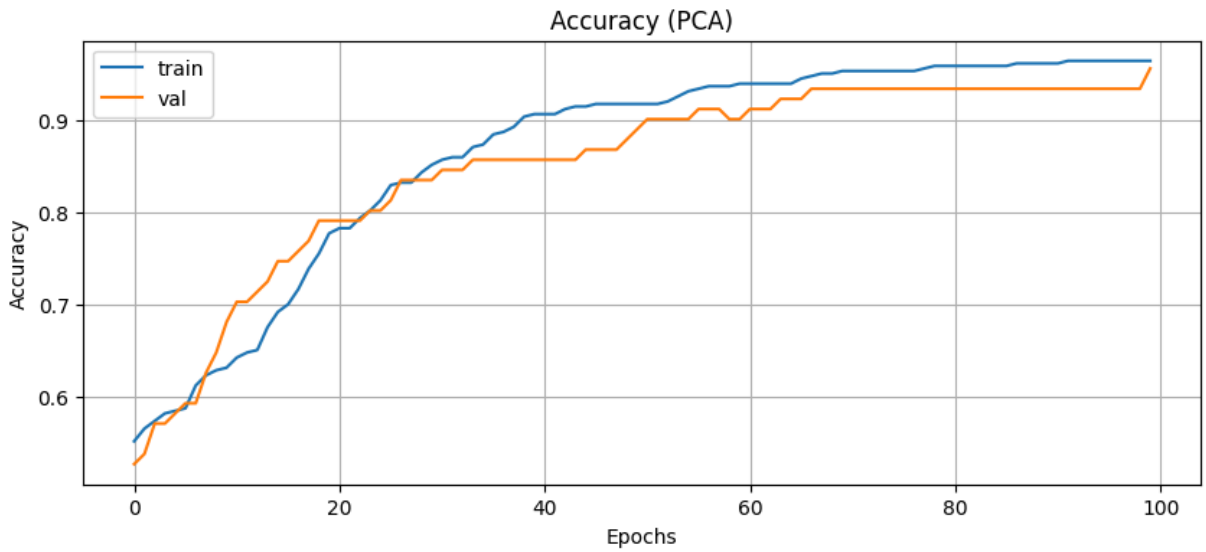
```
Epoch 57/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9360 - loss: 0.2514 - val_accur
acy: 0.9121 - val_loss: 0.2526
Epoch 58/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9360 - loss: 0.2481 - val_accur
acy: 0.9121 - val_loss: 0.2500
Epoch 59/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9360 - loss: 0.2449 - val_accur
acy: 0.9011 - val_loss: 0.2474
Epoch 60/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9393 - loss: 0.2419 - val_accur
acy: 0.9011 - val_loss: 0.2449
Epoch 61/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9393 - loss: 0.2389 - val_accur
acy: 0.9121 - val_loss: 0.2426
Epoch 62/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9393 - loss: 0.2360 - val_accur
acy: 0.9121 - val_loss: 0.2403
Epoch 63/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9393 - loss: 0.2332 - val_accur
acy: 0.9121 - val_loss: 0.2381
Epoch 64/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9393 - loss: 0.2304 - val_accur
acy: 0.9231 - val_loss: 0.2359
Epoch 65/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9393 - loss: 0.2278 - val_accur
acy: 0.9231 - val_loss: 0.2339
Epoch 66/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9472 - loss: 0.2252 - val_accur
acy: 0.9231 - val_loss: 0.2319
Epoch 67/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9513 - loss: 0.2227 - val_accur
acy: 0.9341 - val_loss: 0.2300
Epoch 68/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9590 - loss: 0.2202 - val_accur
acy: 0.9341 - val_loss: 0.2281
Epoch 69/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9590 - loss: 0.2179 - val_accur
acy: 0.9341 - val_loss: 0.2263
Epoch 70/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9623 - loss: 0.2155 - val_accur
acy: 0.9341 - val_loss: 0.2246
Epoch 71/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9623 - loss: 0.2133 - val_accur
acy: 0.9341 - val_loss: 0.2229
Epoch 72/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9623 - loss: 0.2111 - val_accur
acy: 0.9341 - val_loss: 0.2213
Epoch 73/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9623 - loss: 0.2089 - val_accur
acy: 0.9341 - val_loss: 0.2197
Epoch 74/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9623 - loss: 0.2069 - val_accur
acy: 0.9341 - val_loss: 0.2181
Epoch 75/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9623 - loss: 0.2048 - val_accur
```

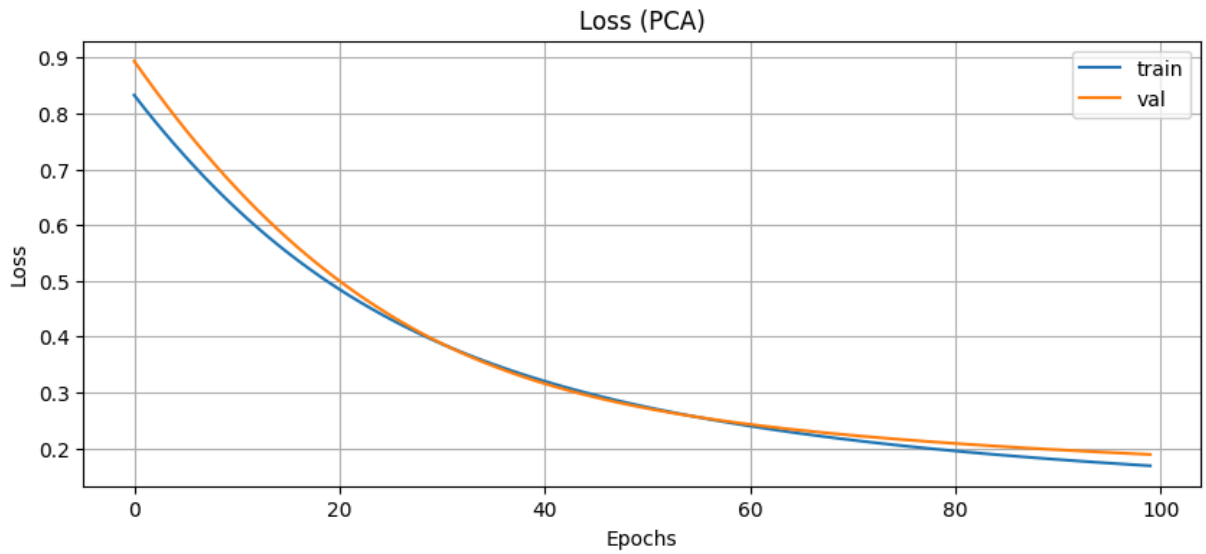
```
acy: 0.9341 - val_loss: 0.2166
Epoch 76/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9623 - loss: 0.2028 - val_accur
acy: 0.9341 - val_loss: 0.2152
Epoch 77/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9623 - loss: 0.2009 - val_accur
acy: 0.9341 - val_loss: 0.2137
Epoch 78/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9656 - loss: 0.1990 - val_accur
acy: 0.9341 - val_loss: 0.2123
Epoch 79/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9664 - loss: 0.1972 - val_accur
acy: 0.9341 - val_loss: 0.2110
Epoch 80/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9664 - loss: 0.1954 - val_accur
acy: 0.9341 - val_loss: 0.2097
Epoch 81/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9664 - loss: 0.1937 - val_accur
acy: 0.9341 - val_loss: 0.2084
Epoch 82/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9664 - loss: 0.1919 - val_accur
acy: 0.9341 - val_loss: 0.2071
Epoch 83/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9664 - loss: 0.1903 - val_accur
acy: 0.9341 - val_loss: 0.2059
Epoch 84/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9664 - loss: 0.1886 - val_accur
acy: 0.9341 - val_loss: 0.2047
Epoch 85/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9664 - loss: 0.1870 - val_accur
acy: 0.9341 - val_loss: 0.2036
Epoch 86/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9664 - loss: 0.1855 - val_accur
acy: 0.9341 - val_loss: 0.2024
Epoch 87/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9676 - loss: 0.1840 - val_accur
acy: 0.9341 - val_loss: 0.2013
Epoch 88/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9676 - loss: 0.1825 - val_accur
acy: 0.9341 - val_loss: 0.2002
Epoch 89/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9676 - loss: 0.1810 - val_accur
acy: 0.9341 - val_loss: 0.1991
Epoch 90/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9676 - loss: 0.1796 - val_accur
acy: 0.9341 - val_loss: 0.1981
Epoch 91/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9676 - loss: 0.1782 - val_accur
acy: 0.9341 - val_loss: 0.1970
Epoch 92/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9753 - loss: 0.1768 - val_accur
acy: 0.9341 - val_loss: 0.1960
Epoch 93/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9753 - loss: 0.1755 - val_accur
acy: 0.9341 - val_loss: 0.1951
Epoch 94/100
```

```
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9753 - loss: 0.1742 - val_accur
acy: 0.9341 - val_loss: 0.1941
Epoch 95/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9753 - loss: 0.1729 - val_accur
acy: 0.9341 - val_loss: 0.1931
Epoch 96/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9753 - loss: 0.1717 - val_accur
acy: 0.9341 - val_loss: 0.1922
Epoch 97/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9753 - loss: 0.1704 - val_accur
acy: 0.9341 - val_loss: 0.1913
Epoch 98/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9753 - loss: 0.1692 - val_accur
acy: 0.9341 - val_loss: 0.1904
Epoch 99/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9753 - loss: 0.1681 - val_accur
acy: 0.9341 - val_loss: 0.1895
Epoch 100/100
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9753 - loss: 0.1669 - val_accur
acy: 0.9560 - val_loss: 0.1886
```

In [27]:

```python
# Gráfica de precisión de entrenamiento y validación a lo largo de las épocas
plt.figure(figsize=(10,4))
plt.plot(history_pca.history['accuracy'], label='train')
plt.plot(history_pca.history['val_accuracy'], label='val')
plt.title('Accuracy (PCA)')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True)
plt.show()

# Gráfica de pérdida de entrenamiento y validación a lo largo de las épocas
plt.figure(figsize=(10,4))
plt.plot(history_pca.history['loss'], label='train')
plt.plot(history_pca.history['val_loss'], label='val')
plt.title('Loss (PCA)')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.grid(True)
plt.show()
```





Interpretación de las curvas (PCA)

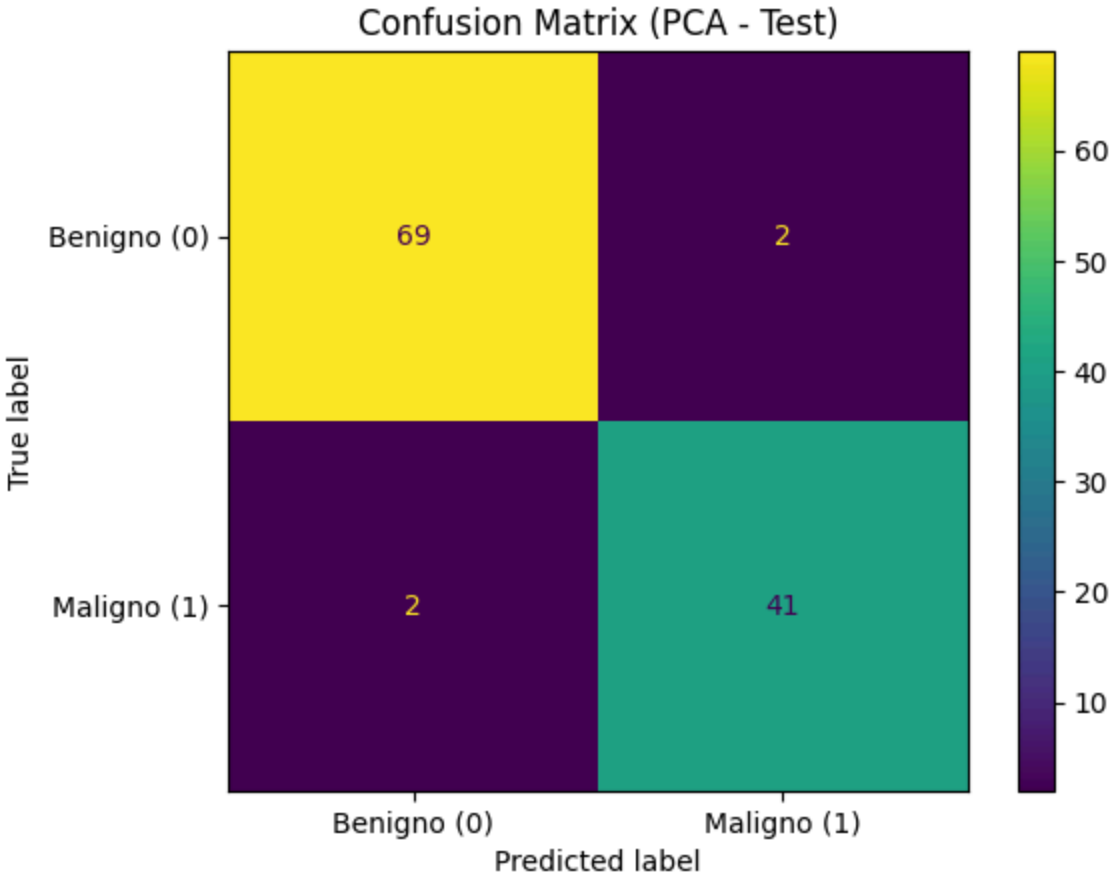
Las curvas muestran que la precisión de entrenamiento y validación aumentan gradualmente hasta valores cercanos a 0.95 y se mantienen bastante cercanas entre sí. Además, la pérdida disminuye de forma constante en ambos conjuntos.

Esto indica que no hay underfitting ni overfitting, ya que el modelo aprende bien y la diferencia entre entrenamiento y validación es pequeña. Por lo tanto, el modelo presenta appropriate fitting

In [28]:
```
#Predecir (probabilidades) y convertir a clases (0 o 1)
y_prob = model_pca.predict(X_test_pca)
y_pred = (y_prob >= 0.5).astype(int).ravel()

#Matriz de confusión
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:\n", cm)
```

```
disp = ConfusionMatrixDisplay(cm, display_labels=["Benigno (0)", "Maligno (1)"])
disp.plot(values_format="d")
plt.title("Confusion Matrix (PCA - Test)")
plt.show()
```

```
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 38ms/step
Confusion Matrix:
 [[69  2]
 [ 2 41]]
```



La matriz de confusión muestra que el modelo clasificó correctamente 69 tumores benignos y 41 malignos, mientras que cometió 4 errores en total: 2 falsos positivos (benignos clasificados como malignos) y 2 falsos negativos (malignos clasificados como benignos).

In [29]:
```
#Métricas
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

TN, FP, FN, TP = cm.ravel()
specificity = TN / (TN + FP)

print("Precision:", round(precision, 4))
print("Recall:", round(recall, 4))
print("Specificity:", round(specificity, 4))
print("F1-score:", round(f1, 4))
```

```
Precision: 0.9535
Recall: 0.9535
Specificity: 0.9718
F1-score: 0.9535
```

Las métricas reflejan un buen desempeño del modelo. La precision (0.9535) indica que aproximadamente el 95.35% de los casos predichos como malignos realmente lo son. El recall (0.9535) muestra que el modelo detecta alrededor del 95.35% de los tumores malignos reales. La specificity (0.9718) indica que el 97.18% de los tumores benignos se clasifican correctamente, evitando la mayoría de los falsos positivos. Finalmente, el F1-score (0.9535) confirma un buen equilibrio entre precisión y recall.

En general, el modelo con PCA presenta un rendimiento alto y equilibrado, aunque con algunos errores de clasificación.

In [30]:
```
model_pca.save("breast_cancer_model_pca.keras")

loaded_model_pca = tf.keras.models.load_model("breast_cancer_model_pca.keras")
loss, acc = loaded_model_pca.evaluate(X_test_pca, y_test, verbose=0)

print("Loaded PCA model accuracy:", round(acc, 4))
```

```
Loaded PCA model accuracy: 0.9649
```

8. Performance Comparison

Metric	Original Model	PCA Model
Precision	1.0000	0.9535
Recall	0.9767	0.9535
Specificity	1.0000	0.9718
F1-score	0.9882	0.9535

Al comparar el modelo con características originales y el modelo con PCA, ambos presentan un desempeño alto. Sin embargo, el modelo original obtiene métricas ligeramente mejores en precisión, recall, especificidad y F1-score.

Las curvas de aprendizaje de ambos modelos muestran un comportamiento similar: la precisión aumenta y la pérdida disminuye de forma estable, con curvas de entrenamiento y validación cercanas. Esto indica que no hay señales de overfitting ni underfitting y que el entrenamiento es estable.

El uso de PCA reduce la dimensionalidad del dataset de 30 variables a 10 componentes, conservando aproximadamente el 95% de la varianza. Esto confirma que existía redundancia en los datos. No obstante, esta reducción provoca una ligera disminución en el desempeño del modelo.

En conclusión, aunque PCA permite simplificar el conjunto de datos y reducir la cantidad de variables necesarias para el entrenamiento, el modelo con características originales presenta el mejor desempeño predictivo. Esto es especialmente importante en un problema médico como el diagnóstico de cáncer, donde es fundamental maximizar la precisión del modelo y minimizar los errores de clasificación. Por esta razón, el modelo con todas las variables originales sería la opción más adecuada para su implementación, mientras que el modelo con PCA podría considerarse en situaciones donde se requiera reducir la complejidad del modelo o mejorar la eficiencia computacional.

9. Personal Conclusions

Carlos.

Se aprendió acerca de las ventajas de usar o no el Principal Component Analysis. Entendimos la utilidad de una red neuronal simple en un problema de clasificación binaria, como en este caso médico donde buscabamos clasificar tumores malignos y benignos.

El PCA fue muy útil para reducir la dimensionalidad del dataset. Originalmente se contaban con 30 características distintas, y gracias a PCA se logró reducir a solo 10 características manteniendo el 95% de la información, mostrando así la redundancia original de la información.

Sin embargo, en un entorno real, no utilizaríamos PCA debido a la pérdida de interpretabilidad. En los entornos médicos es necesario poder explicar con claridad lo que sucede, PCA lo que hace es combinar linealmente distintas características que dejan de representar elementos "reales".

https://github.com/Carlos-HC/Single-Neuron-Network-Team9

Ramón

En este trabajo se desarrolló y evaluó un modelo de red neuronal de una sola neurona para clasificar tumores como benignos o malignos. Durante el proceso se aprendió cómo preparar los datos, entrenar un modelo y evaluar su desempeño.

Se observó que una red neuronal simple puede obtener muy buenos resultados en un problema de clasificación binaria. Además, el uso de PCA permitió reducir el número de variables del dataset, mostrando que existía redundancia en los datos. Sin embargo, también se notó una ligera disminución en el desempeño del modelo al usar PCA.

Como limitación, el modelo utilizado es bastante simple. En el futuro, se podrían probar modelos más complejos o ajustar mejor los parámetros para mejorar el rendimiento.

Este trabajo permitió comprender mejor cómo funcionan los modelos de aprendizaje automático y cómo técnicas como PCA pueden ayudar a simplificar los datos manteniendo

un buen desempeño.

https://github.com/ramonlerma-cr/Single-Neuron-Network-Team9

Bernardo

Esta tarea tenía como objetivo ayudarnos a entender mejor como es que funcióna el modelo de una sola neurona y entender como es que puede PCA afectar al desempeño de esta. Para esto se desarrollaron dos modelos de una sola neurona (sin y con PCA) que identifican y clasifican tumores como malignos o benignos, todo esto utlizando el dataset "Wisconsin Breast Cancer Dataset" en donde hay datos relacionados al diagnóstico de cáncer de mama.

Al final el desempeño de ambos modelos fue bastante bueno. Debido a que PCA elimina variables para simplificar el conjunto de datos este acabó con un desempeño ligeramente peor al que se entrenó con todas las variables, aunque aún así fue capaz de generar muy buenas predicciónes al mismo tiempo que redujo de forma significativa la complejidad del dataset (de 31 a 10 variables). A pesar de esto se opto por seleccionar al modelo sin PCA para ser usado en casos reales ya que al ser una problemática de tanto riesgo como lo puede ser el cáncer de mama hay mucha más prioridad por mantener una predicción lo más correcta posible.

Como conclusión creo que esta tarea me ayudó a entender como funcionan los modelos de una sola neurona, así como identificar los efectos que puede tener el PCA sobre sus predicciónes y desempeño en general.

https://github.com/ber-rv/Single-Neuron-Network-Team9

10. References

UCI Machine Learning Repository. (1995). Breast Cancer Wisconsin (Diagnostic) Dataset. Disponible en: https://archive.ics.uci.edu/dataset/17/breast+cancer+wisconsin+diagnostic